Tictactoe program

K V PANDURANGA RAO • Mar 4

```
win(B,P):-member(L,[[1,2,3],[4,5,6],[7,8,9],[1,4,7],[2,5,8],
[3,6,9],[1,5,9],[3,5,7]]),
maplist({B,P}/[I]>>nth1(I,B,P),L).
replace([_|T],1,X,[X|T]).
replace([H|T],I,X,[H|R]):-I>1,I1 is I-1,replace(T,I1,X,R).
move(B,N,P,Nb):-nth1(N,B,'_'),replace(B,N,P,Nb).
bfs([[B|P]|_],[B|P]):-win(B,x),!.
bfs([[B|P]|R],S):-findall([N,B|P],
(between(1,9,I),move(B,I,x,N)),Ns),
append(R,Ns,Q), bfs(Q,S).
show([A,B,C,D,E,F,G,H,I]):-format('~w|~w|~w~n-+-+-
~n~w|~w|~w~n-+-+-~n~w|~w|~w~n~n',
[A,B,C,D,E,F,G,H,I]).
start:-E=['_','_','_','_','_','_','_','_','_'],bfs([[E]],S),
reverse(S,R),forall(member(X,R),show(X)).
```

Class comments

M

Add class comment...

24 Batch CSE(AI & ML) Minor Course
ECE(A,B&C),ASE

Monkey-banana

K V PANDURANGA RAO • Mar 4

```
goal(state(_,_,_,yes)).
move(state(M,B,onfloor,H), walk(M,B), state(B,B,onfloor,H)) :-
M \= B.
move(state(M,B,onfloor,H), climb, state(M,B,onbox,H)).
move(state(M,B,onbox,no), grasp, state(M,B,onbox,yes)) :- M
= B.
plan(S,[]) :- goal(S).
plan(S,[A|R]) :- move(S,A,S2), plan(S2,R).
start :-
S0 = state(door, window, onfloor, no), plan(S0, Actions),
writeln('Monkey can get banana by:'),
forall(member(X, Actions),
(write('- '), writeln(X))).
```

Class comments

M

Add class comment...

24 Batch CSE(AI & ML) Minor Course
ECE(A,B&C),ASE

TSP

K V PANDURANGA RAO • Mar 4

```
d(a,b,10).
d(a,c,15).
d(a,d,20).
d(b,a,10).
d(b,c,35).
d(b,d,25).
d(c,a,15).
d(c,b,35).
d(c,d,30).
d(d,a,20).
d(d,b,25).
d(d,c,30).
tsp(Start,Path,Cost):- route(Start,
[Start],PathRev,0,Cost),reverse(PathRev,Path).

route(City,Visited,[City|Visited],Cost,Total):-
length(Visited,4), d(City,a,Back),
Total is Cost+Back.

route(City,Visited,Path,Cost,Total):-
findall((Next,Dist),(d(City,Next,Dist),
\+member(Next,Visited)),List),
List \= [],
sort(2,@=<,List,[(Nearest,D)|_]),
NewCost is Cost+D,
route(Nearest,[Nearest|Visited] ,Path,NewCost,Total).
```

Class comments

M Add class comment...

Hill Climbing.cpp

K V PANDURANGA RAO • Mar 11

```
#include <bits/stdc++.h>

using namespace std;

vector<vector<int>> g={{1,2,3},{4,5,6},{7,8,0}};int dx[4]=
{1,-1,0,0},dy[4]={0,0,1,-1};

int h(auto s){int c=0;for(int i=0;i<3;i++)for(int j=0;j<3;j++)if(s[i]
[j]&&s[i][j]!=g[i][j])c++;return c;}

pair<int,int> z(auto s){for(int i=0;i<3;i++)for(int
j=0;j<3;j++)if(!s[i][j])return{i,j};return{-1,-1};}

int main(){vector<vector<int>> s={{1,2,3},{4,0,6},{7,5,8}};int
c=h(s);

while(1){auto[x,y]=z(s);auto b=s;int bc=c;

for(int k=0;k<4;k++){int nx=x+dx[k],ny=y+dy[k];

if(nx>=0&&ny>=0&&nx<3&&ny<3){auto t=s;swap(t[x][y],t[nx]
[ny]);int nc=h(t);

if(nc<bc){bc=nc;b=t;}}}

if(bc>=c)break;s=b;c=bc;}

for(auto&r:s){for(int v:r)cout<<(v?to_string(v):"_")<<"
";cout<<"\n";}}
```

Class comments

M

Add class comment...

24 Batch CSE(AI & ML) Minor Course
ECE(A,B&C),ASE

min-max.py

K V PANDURANGA RAO • Mar 11 (Edited Mar 11)

```
import math

board = [' ' for _ in range(9)]

def check_winner(b, p):
    # Standard win conditions
    wins = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),
(2,4,6)]
    return any(b[i] == b[j] == b[k] == p for i, j, k in wins)

def minimax(b, is_max):
    # Terminal states
    if check_winner(b, 'X'): return 1
    if check_winner(b, 'O'): return -1
    if ' ' not in b: return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X' # Make move
                best = max(best, minimax(b, False))
                b[i] = ' ' # Undo move
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O' # Make move
                best = min(best, minimax(b, True))
                b[i] = ' ' # Undo move
        return best
```

```
    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X' # Make move
                best = max(best, minimax(b, False))
                b[i] = ' ' # Undo move
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O' # Make move
                best = min(best, minimax(b, True))
                b[i] = ' ' # Undo move
        return best

def best_move():
    move, best_val = -1, -math.inf
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            val = minimax(board, False)
            board[i] = ' '
            # FIXED: Comparison is now inside the move validation
check
            if val > best_val:
                best_val, move = val, i
    return move

# Test case
board[0]='O'; board[4]='X'; board[8]='O'
m = best_move()
board[m] = 'X'
print(f"AI chooses index: {m}")
print(board)
```

Class comments

M

Add class comment...

24 Batch CSE(AI & ML) Minor Course
ECE(A,B&C),ASE

unification.py

K V PANDURANGA RAO • Mar 11

```
def unify(x,y,subs={}):
if subs is None: return None
if x==y: return subs
if isinstance(x,str) and x[0].islower(): return
unify_var(x,y,subs)
if isinstance(y,str) and y[0].islower(): return
unify_var(y,x,subs)
if isinstance(x,tuple) and isinstance(y,tuple) and
len(x)==len(y):
for a,b in zip(x,y):
subs=unify(a,b,subs)
if subs is None: return None
return subs
return None
def unify_var(v,val,subs):

if v in subs: return unify(subs[v],val,subs)
if val in subs: return unify(v,subs[val],subs)
subs[v]=val; return subs
x=('father','john','X'); y=('father','Y','mary')
print("Substitution:", unify(x,y,{}))
```

Class comments

M

Add class comment...

24 Batch CSE(AI & ML) Minor Course
ECE(A,B&C),ASE

resolution.py

K V PANDURANGA RAO • Mar 11

```
clauses=[{'~Rain','Wet'},{'Rain'},{'~Wet'}]
def resolve(c1,c2):
for l in c1:
if l.startswith('~') and l[1:] in c2: return (c1-{l})|(c2-{l[1:]})
if not l.startswith('~') and ('~'+l) in c2: return (c1-{l})|(c2-
{'~'+l})
return None
def resolution(clauses):
new=set()
while True:
n=len(clauses)
for i in range(n):
for j in range(i+1,n):
r=resolve(clauses[i],clauses[j])
if r==set(): return True
if r and r not in clauses: new.add(frozenset(r))
if not new: return False
clauses.extend([set(c) for c in new]);
new.clear()
print("Contradiction found:", resolution(clauses))
```

Class comments

M

Add class comment...